

Data management of SmartSolo node-type geophones using open-source Python

Tobias Stål  * 1

¹University of Tasmania, Hobart, Tasmania, Australia

Author contributions: *Conceptualization*: Tobias Stål. *Methodology*: Tobias Stål. *Software*: Tobias Stål. *Validation*: Tobias Stål. *Investigation*: Tobias Stål. *Data Curation*: Tobias Stål. *Visualization*: Tobias Stål. *Writing - Original draft*: Tobias Stål.

Abstract Autonomous three-component nodes such as the SmartSolo IGU-16HR are now common in passive seismology, but their data arrive in an undocumented binary format and with state-of-health logs that are usually read through vendor software on Windows. We present `node_toolbox`, an open-source Python package that reads every file a SmartSolo node writes. It parses the logs into a time-indexed pandas table, derives one deployment per power-up from the first stable GPS position, and selects deployments by radius, polygon and time. It reads the raw DLD data files directly, without vendor software, and cuts the selected windows into MiniSEED or SEG-Y with StationXML metadata, polarity correction and instrument response. It also analyses the geophone pulse test and the compass and tilt records. While documenting the DLD format we found that the time text in each data block is 2 s late until the GPS receiver has learned the leap seconds; the GPS time of week in the same block is not affected, and the reader uses it. We verify the package with unit tests, two Antarctic nodes and a six-node vehicle braking test. The repository contains the code, tests, example data and tutorial notebooks.

Non-technical summary Small, self-contained seismic sensors called nodes are planted in large numbers to record ground vibrations from earthquakes, ice and traffic. Each node stores its recordings and a diary of its health: temperature, battery, GPS position and tilt. For one widely used brand, SmartSolo, these files are normally opened with the manufacturer's Windows program. We wrote free Python software that reads all of these files directly. It works out where and when each sensor was recording and cuts out the data for chosen places and times. It saves them in the standard formats seismologists share, together with a description of each sensor. It also checks whether the sensors worked properly. We found and corrected a hidden 2-second timing error in how the files label time. We tested the software on sensors left for two weeks on the Antarctic ice and on six sensors beside a road in Hobart, Australia, where a four-wheel-drive car braked repeatedly.

*Corresponding author: tobias.staal@utas.edu.au

1 Introduction

Autonomous geophone nodes, which combine a 5 Hz three-component sensor, a 24-bit digitizer, a GNSS receiver, a battery and storage in one sealed unit, have made dense passive arrays practical (Karplus and Schmandt, 2018; Ringler et al., 2018). Their ease of deployment is especially valuable where logistics are constrained, such as on the Antarctic ice sheet, where every kilogram flown and every minute spent at a site counts. The SmartSolo IGU-16HR 3C (DTCC) is among the most widely used of these instruments (Zeckra et al., 2022).

Deploying many nodes, however, shifts the work to data management. A SmartSolo node stores its raw samples in an undocumented binary format (`.DLD`), and writes its state of health to a text log (`DigiSolo.LOG`) whose content changes between firmware versions. The usual workflow is to harvest and export the data with the vendor's Windows software (SoloLite). The positions, orientation and quality-control information are then transferred by hand into station metadata. Several pitfalls in this chain are known. All IGU-16HR channels have negative polarity relative to the FDSN convention. A missing leap-second configuration file leads to an 18-s timing error at export. The programmed preamplifier gain must be removed consistently (AusPass, 2025b,a). Field crews also need to know when a node was moved, tilted or tested while being handled, which is recorded in the logs but rarely used.

Here we describe `node_toolbox` (Stål, 2026), a pure-Python package for SmartSolo node data built on ObsPy (Beyreuther et al., 2010; Krischer et al., 2015), pandas (McKinney, 2010), NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), GeoPandas (Jordahl et al., 2020) and Matplotlib (Hunter, 2007). The package (i) parses state-of-health logs from all firmware versions we have seen into one time-indexed table, (ii) derives deployments and selects them in space and time, (iii) reads the raw `.DLD` files directly, (iv) writes MiniSEED, SEG-Y (SEG Technical Standards Committee, 2017) and StationXML (International Federation of Digital Seismograph Networks, n.d.) with polarity, gain and response handled, and (v) provides sensor quality control from the pulse test, boot-time geophone tests, compass and tilt. We document the `.DLD` format, including a timing behaviour that affects any reader of these files. We then verify the package on two Antarctic nodes and a six-node braking test.

2 Instrument and files

Each node writes the files listed in Table 1. Harvesting with SoloLite copies them into a folder named after the serial number and harvest time, but `node_toolbox` does not depend on that layout. Serial numbers are taken from file contents where possible, otherwise from paths. The examples in this paper come from nodes with firmware V1.0.5.6kp, V1.0.8.1be, V1.1.2.0be and V1.1.4.2be recording at 250, 500 and 1000 samples per second (sps).

3 Software design

`node_toolbox` consists of six modules (Fig. 1; about 3000 lines of Python). They exchange plain data structures rather than custom classes. Logs become a pandas DataFrame with a UTC time index. Deployments become a GeoDataFrame, waveforms an ObsPy `Stream` and metadata an ObsPy `Inventory`. A user can therefore stop at any stage and continue with standard tools, for example to plot the temperature of every node with one line of pandas. All times are UTC. Every function that guesses (a position, a time convention, a polarity) records what it did in a table column, so that the choice can be audited.

File	Content	Module
DigiSolo.LOG	state-of-health log: boots, GPS, temperature, battery, memory, compass/tilt, boot-time geophone test	smartsolo_log, smartsolo_locate, smartsolo_orientation
seisNNN{X,Y,Z}.DLD	raw samples, one file per component and power-up, with GPS time and position tags	smartsolo_dld
PULSE_{X,Y,Z}.WAV	geophone pulse test of the latest power-up	smartsolo_node
sct_par.xml, SCT_INT.XML	acquisition script with test limits	smartsolo_node
device.ini	serial number and firmware (not on all nodes)	smartsolo_node
SoloLite MiniSEED/SEG-Y	optional exported waveforms	smartsolo_waveforms

Table 1 Files written by a SmartSolo IGU-16HR node and the node_toolbox module that reads them.

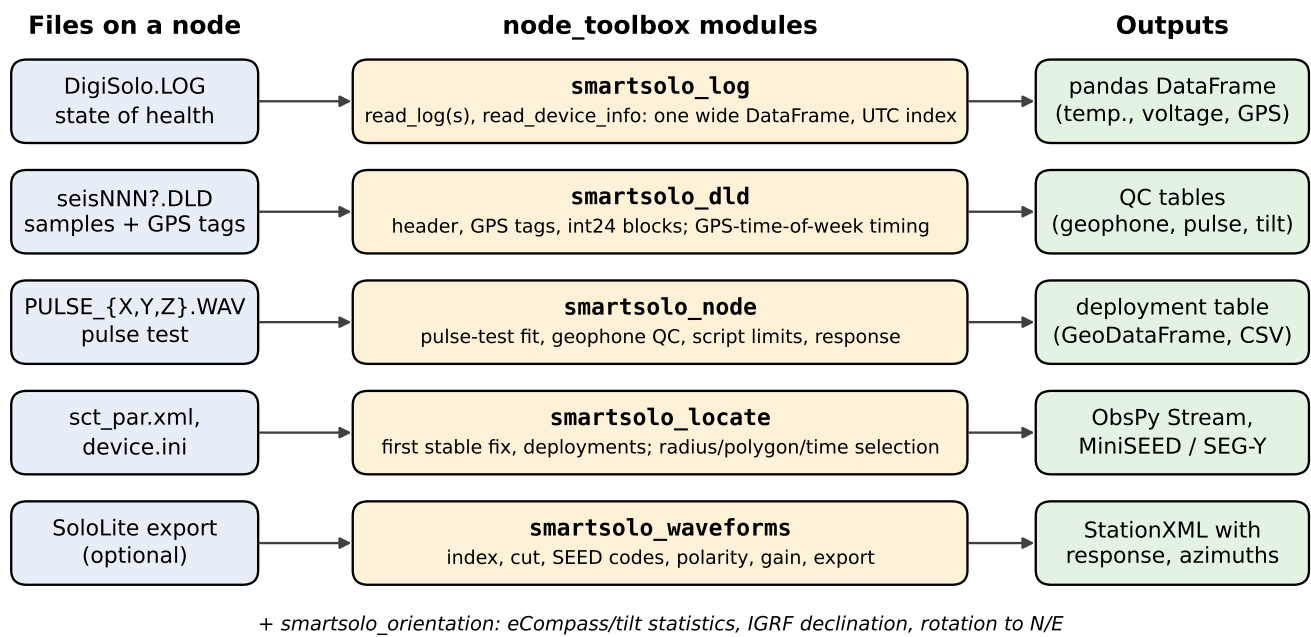


Figure 1 Structure of node_toolbox. Files written by a node (left) are read by the modules (centre), which return pandas, GeoPandas and ObsPy objects and standard files (right).

3.1 State-of-health logs

DigiSolo.LOG is an INI-like text file of numbered sections (e.g. [DeviceInfo00001], [GPS00042]). Its first line lists the number of DeviceInfo, GPS, Temperature, Memory, Battery, Error and Notify records. smartsolo_log.read_log turns each section into one row of a single wide DataFrame, one column per key in snake case, indexed by the record's UTC time. The record type, record number, line number and power-up session number are kept as columns. This layout keeps every piece of information while making simple questions simple (Listing 1).

The parser handles firmware differences found in our files:

- Firmware V1.1.4 writes [GNSS...] sections and GNSS ... keys, which are mapped to GPS.
- UTC Time = ", " before the first fix and Altitude = Unknown are read as missing values.
- Comma-separated tuples of 2–4 numbers (e.g. two coil resistances, three ADC synchronisation values) are split into numbered columns.
- V1.0.5 logs one coil resistance instead of two, and an eCompass heading in $-180...180^\circ$ instead of $0...360^\circ$.

- Boots without a real-time-clock time, and pre-lock times in 2015/2016, are flagged rather than dropped.

Times that must be inferred (`[BatteryPowerOnStop]` has none) are marked in a `time_inferred` column. The parsed record counts are compared with the header counts. They agree for all record types except Notify, whose header count is often off by a few.

One key needs interpretation. The logged `Sample Rate` is not a rate but the sample interval in units of $10\ \mu\text{s}$:

$$f_s = \frac{10^5}{S} \text{ sps}, \quad (1)$$

so $S = 100, 200$ and 400 correspond to $1000, 500$ and 250 sps. We confirmed this against the time tags of the raw data (Section 3.3), and the parser adds the derived `sample_rate_hz`. Without this correction the Antarctic nodes would be catalogued at 100 sps instead of 1000 sps.

Listing 1 Reading logs and plotting state of health.

```
import smartsolo_log as sl
df = sl.read_logs("data/nodes")
df.groupby("serial").temperature.plot()
info = sl.read_device_info("data/nodes")
```

3.2 Deployments and selection

A node may be switched on several times: on the bench, in transit and at one or more sites. The function `build_deployments` splits the log into one deployment per power-up. It places each deployment at the first stable GPS position, defined as the first run of $n = 5$ consecutive fixes that all lie within $r = 10$ m of their median. Cold-start fixes before that run are counted (`unstable_fixes_skipped`) but not used. A later power-up somewhere else becomes a separate deployment, so a node that was moved never contaminates the position of its first site. For each deployment the table also records the median and maximum offset of later fixes, the drift between the first and last day, the firmware, sample rate, gain, and summary values of tilt, heading and the boot-time geophone test.

Because the raw data files carry a GPS position every 1000 samples (Section 3.3), `build_deployments_from_dld` builds the same table from `.DLD` files alone, and `combine_deployments` merges both sources. These tag positions also show when nodes were picked up while still recording (Fig. 7).

`select_deployments` filters deployments by a circle (point and radius in km), by a polygon (any GeoPandas-readable file or Shapely geometry, using a spatial join), and by a time window. It returns the overlap of the window with each deployment (Listing 2). `export_stations` writes the selection as CSV or GeoJSON.

Listing 2 Selecting and cutting waveforms.

```
import smartsolo_locate as loc
import smartsolo_waveforms as wf
deps = loc.build_deployments_from_dld(
    "data/break_test")
sel = loc.select_deployments(
    deps, point=(-42.9017, 147.3301),
```

```

104     radius_km=0.03,
105     start="2023-03-31T01:40",
106     end="2023-03-31T01:41")
107 idx = wf.index_waveforms("data/break_test")
108 st = wf.extract_waveforms(
109     sel, idx, out_dir="out",
110     out_format="MSEED")
111 inv = wf.build_inventory(
112     sel, stream=st, response="auspass")

```

113 3.3 Raw DLD files

114 The .DLD format is undocumented. We inferred its layout only from data files recorded by our own nodes. We com-
115 pared byte patterns with the values in the nodes' logs and checked the result against the physical signal; no vendor
116 software was decompiled or disassembled. The layout is written down byte by byte in the repository (`docs/DLD_FORMAT.md`),
117 together with a statement of how it was derived (`docs/CLEAN_ROOM.md`), and `smartsolo_dld` was written from that
118 description.

119 A file consists of a 512-byte header followed by blocks of 1000 samples, each followed by a 72-byte tag (Fig. 2a).
120 The header begins with the magic string `DTCCSZ-TEC-FTS` and holds the serial number, firmware, project name,
121 start and end times, sample interval and a leap-second field at byte 0x104. It also holds a position, which is written
122 when the file is closed and is therefore the position at pick-up, not at installation. Samples are 24-bit little-endian
123 two's-complement integers in raw ADC counts, including the preamplifier gain. The samples are continuous across
124 tags (Fig. 2b). Each tag contains a millisecond tick whose step gives the block duration and hence the sample rate
125 (Eq. 1), the UTC date and time as text, latitude and longitude, a small phase value, a counter and the GPS time of week
126 (TOW) as text. The counter is the time since the last GPS synchronisation in units of 10 ms. It increases by 100 per
127 second and resets at each synchronisation, about every 9 minutes in our files (Fig. 2c).

128 **Timing.** The time text in each tag looks like the obvious time stamp, but it is not reliable at the 2-s level. In files
129 whose header leap-second field is 0, the text time is exactly 2 s later than the time derived from the TOW. In files
130 whose field is 18, the two agree. Files that span the moment the receiver learns the current GPS-UTC offset show the
131 text time jumping back by 2 s in mid-file, for example a repeated `00:58:37` after block 2197 in one of our files. We
132 interpret this as the receiver using an outdated default offset of 16 s until the almanac delivers the current value of
133 18 s. The TOW is continuous across such jumps. As in common GPS timing receivers, it is the time of the next pulse
134 per second. `node_toolbox` therefore computes the UTC time of a tag as

$$135 \quad t_{\text{UTC}} = t_0 + 604800 W + \frac{\text{TOW}}{1000} - 1 - \Delta t_{\text{LS}}, \quad (2)$$

136 where t_0 is the GPS epoch (1980-01-06), W the GPS week (from the tag date), TOW in ms, and Δt_{LS} the GPS-UTC
137 leap-second offset at that time (18 s since 2017). This is the default (`time_source="tow"`). The text time remains
138 available, and the difference between the two is reported for every tag (`label_minus_tow_s`). Section 4.3 shows the

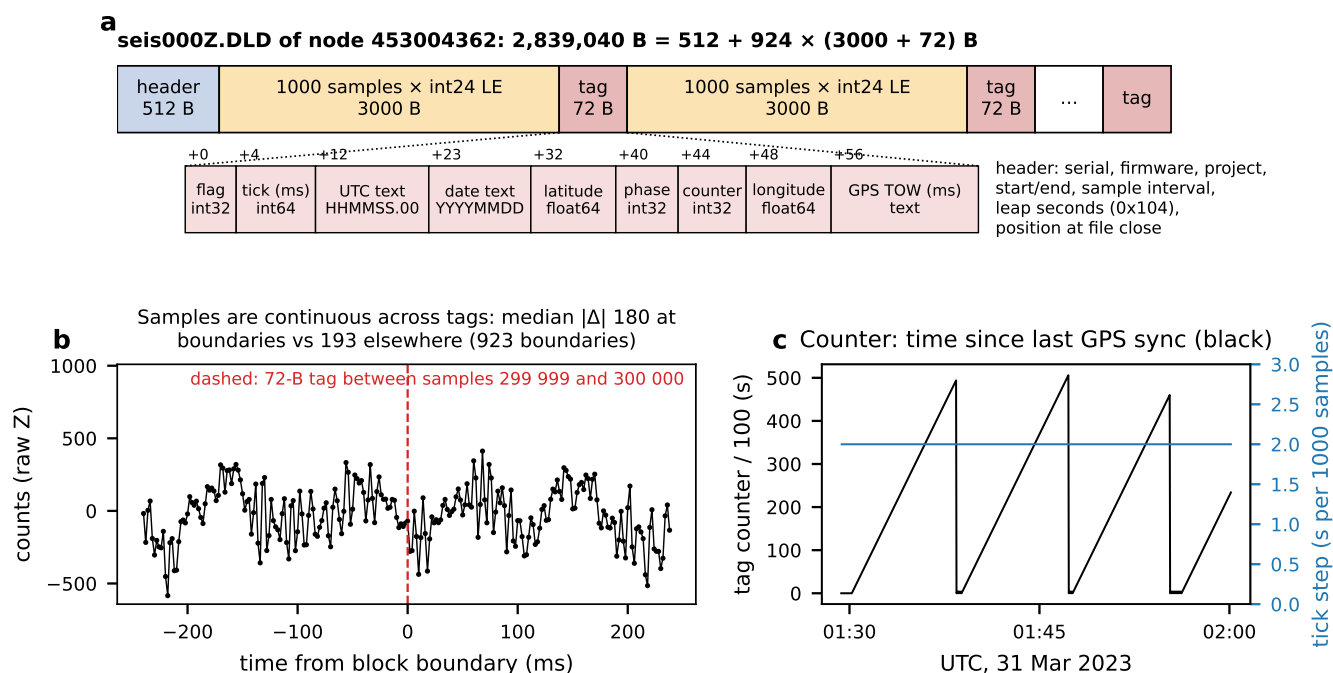


Figure 2 The DLD format. (a) File layout and the 72-byte tag after every 1000 samples (byte offsets within the tag). (b) Raw samples around a block boundary of node 453004362: the tag interrupts the byte stream but not the signal; the median absolute sample-to-sample difference at the 923 boundaries equals that elsewhere. (c) Tag counter (time since the last GPS synchronisation) and tick step (2 s per 1000 samples, i.e. 500 sps).

effect between nodes.

One convention remains unverified: whether a tag time refers to the first sample of the block before it (default, `tag_marks="block_start"`) or to the sample after the block. The alternative shifts all samples by one block (1, 2 or 4 s). Because the time of the first sample cannot be cross-checked between nodes, this needs one comparison with a SoloLite export of the same file. `compare_with_export` does this comparison.

`read_dld` returns an ObsPy Stream for any time window, reading only the blocks needed. Gaps (a change of tick step or a time jump) start a new trace. The function `read_dld_node` reads the three components of a power-up, `scan_dld` lists the continuous segments without reading samples, and `convert_dld` converts a whole drive. Files are opened read-only and are never modified, which a unit test confirms.

3.4 Waveforms, codes and metadata

`index_waveforms` builds a table of all waveform files under a directory, including .DLD, MiniSEED and SEG-Y, with serial, component, start and end. It can cache the table as CSV. `extract_waveforms` cuts the selected window for each selected deployment, merges and trims it, assigns SEED codes ([International Federation of Digital Seismograph Networks, 2012](#)) and writes MiniSEED or SEG-Y files with one file per channel and window. Window ends are exclusive, so consecutive windows do not share samples.

Codes. Network, station and location codes come from an optional mapping table (serial to codes, optionally with validity times). Without a row, the station code defaults to the last five digits of the serial. The band code follows the sample rate (D at 250 and 500 sps, G at 1000 sps), the instrument code is P (short-period geophone), and the node axes X and Y map to N and E ([DTCC, n.d.](#)), giving, e.g., DPZ, DPN, DPE.

Polarity. The SmartSolo manual defines positive output for case motion to the south, west and down (DTCC, n.d.). Controlled tests by AusPass confirm negative polarity for all three IGU-16HR channels relative to FDSN convention (AusPass, 2025a). `extract_waveforms` therefore multiplies IGU-16HR data by -1 by default (`invert_polarity="auto"`). It records this in the trace header (`stats.processing`), and `build_inventory` writes the channel orientation to match.

Gain and response. Raw DLD counts include the preamplifier gain G (in dB, from the log). The AusPass response for gain-removed data has zeros at 0 (twice), poles at $p_{1,2} = -22.211059 \pm 22.217768 i$ rad/s (5 Hz, damping 0.707) and a flat high-frequency sensitivity of 257 019 225.55 counts/(m/s) (AusPass, 2025b). `build_inventory(response="auspass")` writes

$$H(s) = S_0 10^{G/20} \frac{s^2}{(s - p_1)(s - p_2)}, \quad (3)$$

with S_0 the AusPass sensitivity and G taken from each deployment (`gain_db="auto"`), or 0 for exports with the gain already removed. Alternatives are a nominal response from the boot-test values or no response. Positions, elevations, sample rate, azimuth and dip complete each channel. StationXML is written through ObsPy.

3.5 Sensor quality control and orientation

Boot-time geophone test. At every power-up the node measures coil resistance, natural frequency f_0 , damping h , sensitivity and noise for each channel and logs a pass flag. `smartsolo_node.geophone_qc` compares the values with the limits in the node's own script (`sct_par.xml`). It also flags tests whose spread noise exceeds a threshold (default 5000). A test taken while the node is being moved can be logged as passed although its damping and sensitivity are out of specification.

Pulse test. `PULSE_{X,Y,Z}.WAV` contains 16 000 samples of 24-bit data whose header claims 10 kHz. The content is a 16-s sequence of current steps followed by a quiet section. This matches the geophone/ADC test stage described in the manual (DTCC, n.d.) at a fixed rate of 1000 sps, independent of the acquisition rate. `analyse_pulse` finds the steps and fits a damped oscillation

$$x(t) = A e^{-h\omega_0 t} \sin(\omega_0 \sqrt{1 - h^2} t + \phi) + C, \quad \omega_0 = 2\pi f_0, \quad (4)$$

to the 0.9 s after each switch-off. It measures the noise floor in the quiet section and converts counts to volts with 3355.4428 counts/mV at 0 dB (DTCC, n.d.).

Orientation. The eCompass heading, tilt, roll and pitch are logged regularly. `smartsolo_orientation` provides circular statistics and separates boot-time readings, taken before the node is planted, from the settled heading. It measures rotation over a deployment and computes the IGRF-13 (Alken et al., 2021) declination and field at each site. It can write azimuths to StationXML or rotate data to north and east. Because it is not documented whether `eCompass North` is a magnetic azimuth of the node's arrow, nor whether the compass was calibrated, the package reports both the raw heading and the heading corrected for declination, and leaves the choice to the user.

4 Verification and test cases

The repository includes 14 unit tests (pytest). They cover:

- record counts against log headers for all bundled logs and a V1.1.4 GNSS log;
- deployment splitting for a synthetic node that was moved;
- radius, polygon and time selection;
- DLD block decoding, sample-rate detection and continuity across tags;
- the 2-s label offset;
- alignment of neighbouring nodes;
- the read-only guarantee;
- a round trip from DLD to MiniSEED and SEG-Y that checks the exported data are exactly -1 times the raw counts.

Five Jupyter notebooks reproduce the examples below and serve as tutorials.

4.1 State of health on the Antarctic ice

Two nodes (453021267 and 453022522, firmware V1.0.8.1be, 1000 sps, 0 dB) were deployed at 1644–1649 m elevation in Dronning Maud Land, Antarctica (71.55°S, 11.12°E) from 25 December 2024 to 8 January 2025. Their logs (2.1 MB each, about 9900 records) also contain bench and transport boots from November 2024. The parser separates these into their own sessions. Figure 3 shows four of the logged quantities against time.

- **Temperature** follows a daily cycle between -17.4 and $+7.7^{\circ}\text{C}$.
- **Battery voltage** falls from 8.25 to 7.63 V over 13.8 days at 1000 sps.
- **Tilt** is constant at 1.7° and 5.1° ; the second node exceeds the 3° specification for the horizontal geophones.
- **Heading** is stable to 0.9° (circular standard deviation). Node 453021267 nevertheless rotates by 2.3° over the deployment ($0.17^{\circ}/\text{day}$) at constant tilt, which points to ice motion or compass drift.

The IGRF declination at the site is -29.5° (inclination -61.3° , horizontal field 19 000 nT).

The first-stable-fix rule matters in practice (Fig. 4). For 453022522 the first two fixes after the cold start were 45–100 m from the final position and are skipped. All later fixes scatter by a few metres. The median position moves by 2.6 and 3.1 m over two weeks, about 0.2 m/day , which is within GPS noise but consistent with ice flow.

4.2 Pulse test and boot-time test

Figure 5 shows the pulse file of 453022522 and the fit of Eq. 4 after a switch-off. Across 21 channels of seven nodes (one Antarctic node and the six nodes of the braking test), the fitted f_0 and h agree with the values the node logged at the same boot, with median absolute differences of 0.06 Hz and 0.011. The noise floor of the quiet section is 1.13– $1.26\text{ }\mu\text{V}$, equal to the logged RMS noise. This confirms both the 1000-sps rate of the pulse file and the counts-per-volt conversion. The pulse file of the braking-test nodes, which recorded at 500 sps, is also at 1000 sps.

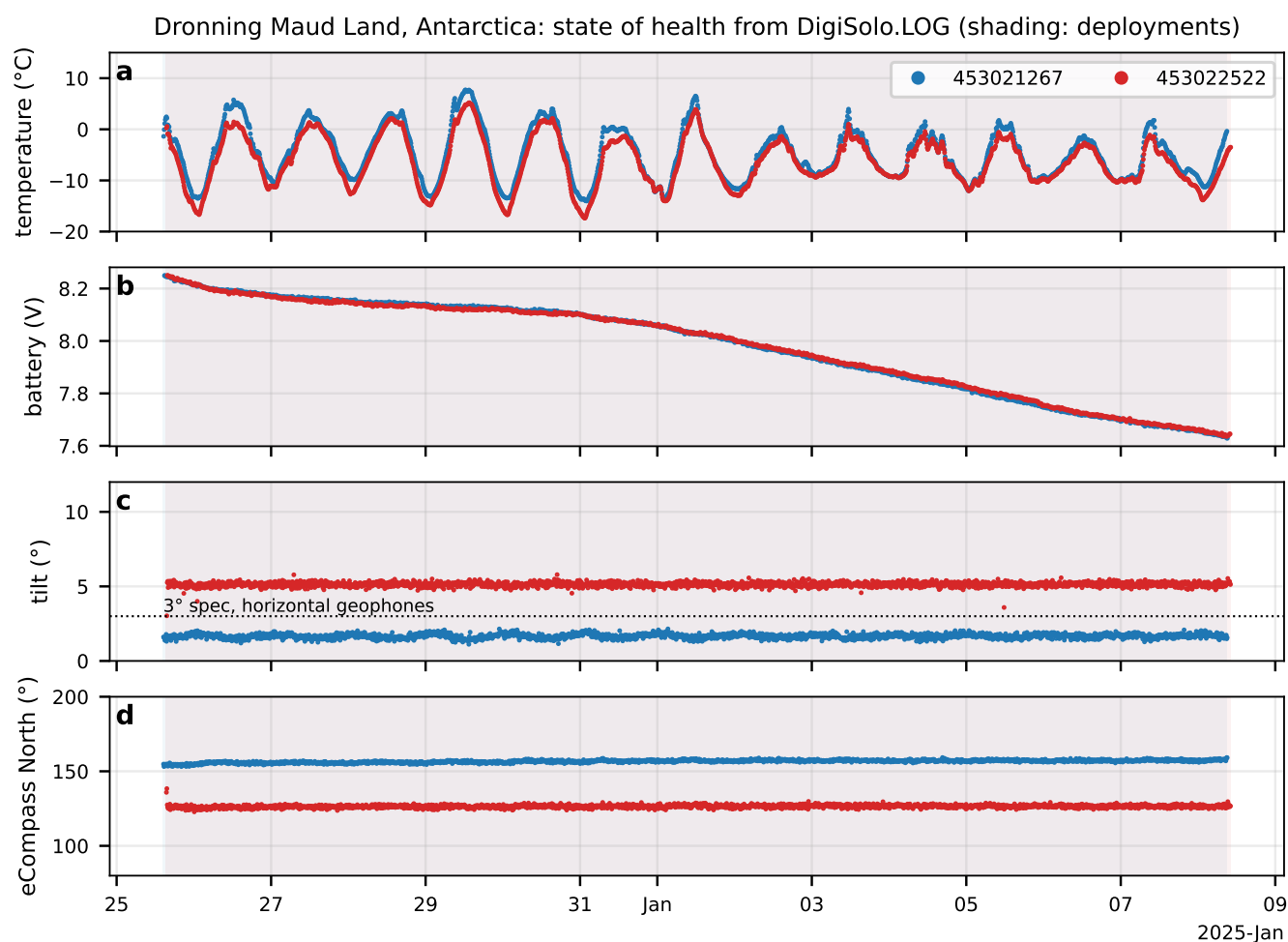


Figure 3 State of health of two nodes in Dronning Maud Land read with `read_logs`: (a) temperature, (b) battery voltage, (c) tilt with the 3° horizontal-geophone specification, (d) eCompass heading. Shading marks the deployments found by `build_deployments`.

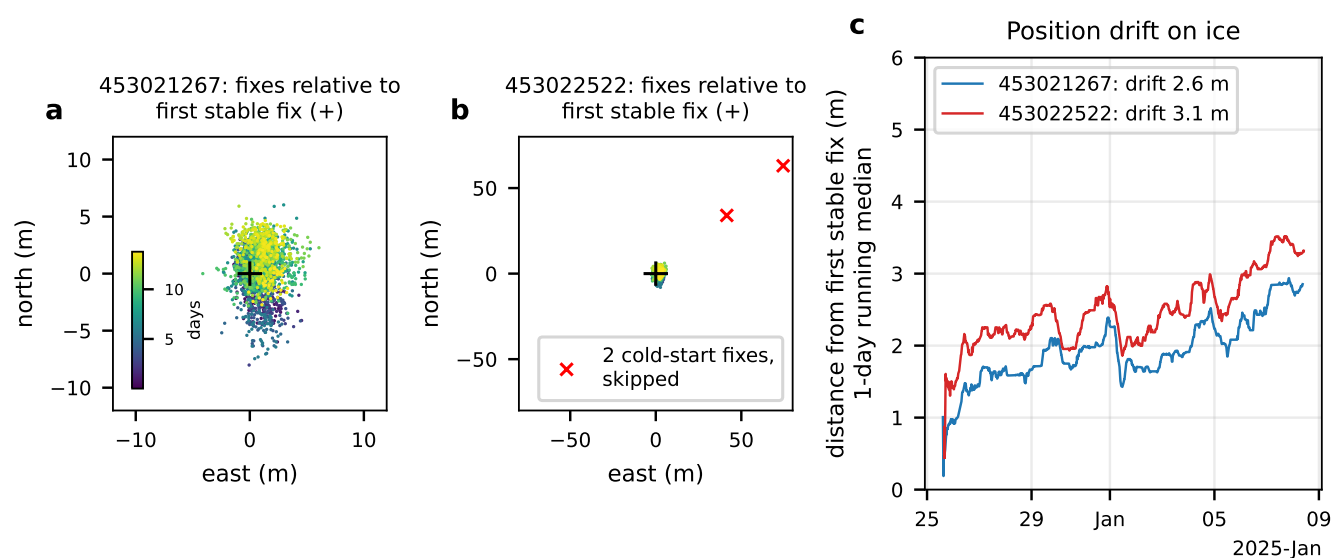


Figure 4 GPS positions in the logs of the two Antarctic nodes relative to the first stable fix (a, b; colour: days after power-up; crosses: cold-start fixes that are skipped) and 1-day running median distance from the first stable fix (c).

- One Antarctic node and one braking-test node tested themselves while being carried (spread noise up to 22 000 and 144 000). Both logged a pass although their damping and sensitivity were out of limits.
- Another node returned an implausible f_0 of 78 kHz on one channel without being noisy, but tested normally at its next boot.
- Coil resistance follows temperature as expected for copper: about 1620 Ω at -2°C and 1730–1780 Ω at room temperature.

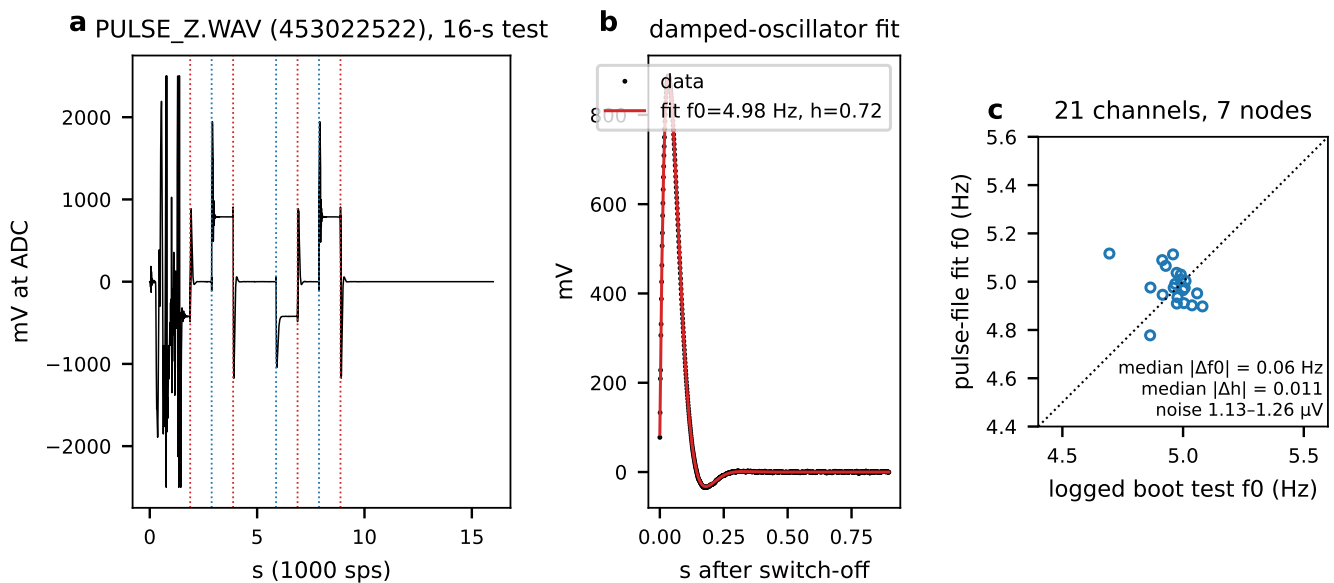


Figure 5 Pulse test. (a) PULSE_Z.WAV of node 453022522 read at 1000 sps (dotted lines: detected current steps). (b) Damped-oscillator fit (Eq. 4) after a switch-off. (c) Natural frequency from the pulse file against the value logged by the node at the same boot, 21 channels of seven nodes.

4.3 Timing between nodes

The 2-s label offset is easy to miss because nodes deployed together usually share it. In all six braking-test files the header leap-second field is 0 and the text time is exactly 2 s ahead of Eq. 2 in every tag (Fig. 6c). Relative timing between those nodes is therefore unaffected.

Problems appear when nodes in one dataset differ. In a longer recording of four of the same nodes (not included in the repository), the text times jumped back by 2 s at different times on different nodes (23:56, 00:21, 00:58 and 02:38 UTC). Cross-correlating ambient noise between node pairs every 10 minutes gave lags of exactly ± 2 s whenever one node's text time had jumped and the other's had not. With TOW times the lags were 0 ± 4 ms throughout. We saw the same 2-s offset in files from firmware V1.1.2 at 250 sps. Two of those nodes, 10 m apart, started 4 s apart, yet their records aligned to within one sample at a correlation of 0.96. This confirms the block decoding and the TOW timing independently of the braking test.

Within the braking test, neighbouring nodes 5 and 12 m apart give median lags of -6 and $+30$ ms in 30-s windows (Fig. 6b). The scatter of individual windows reflects the moving vehicle sources rather than timing.

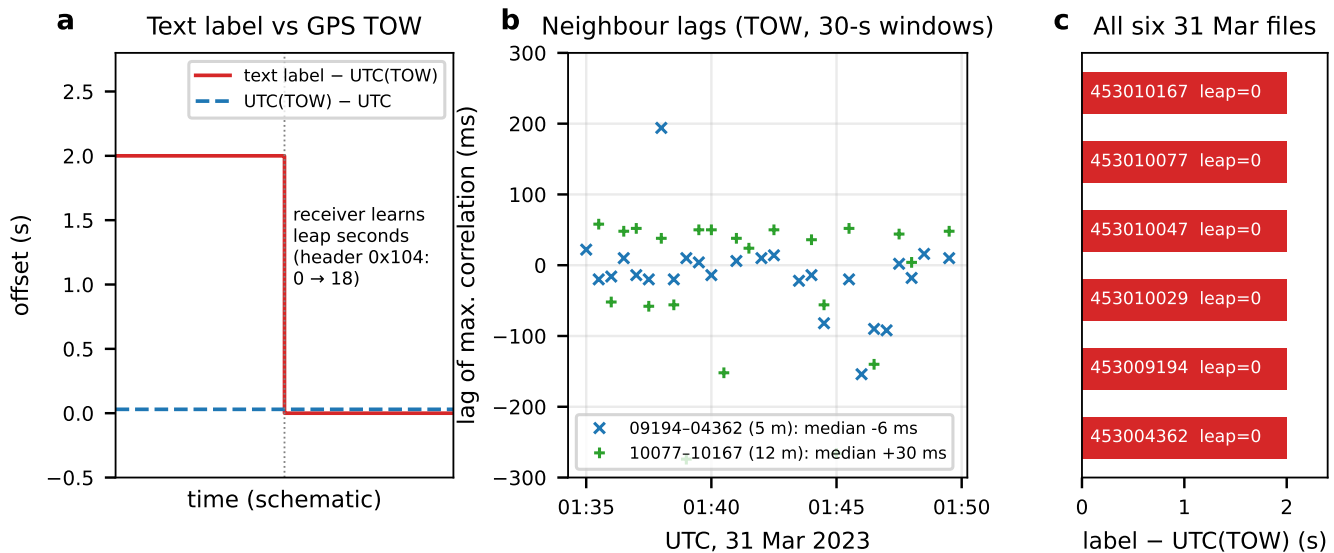


Figure 6 Timing. (a) Schematic: the text time in each tag is 2 s late until the GPS receiver knows the current leap seconds, while the GPS time of week (TOW, Eq. 2) is not affected. (b) Lag of maximum cross-correlation between neighbouring nodes using TOW times. (c) Text time minus TOW-derived UTC and the header leap-second field for all six braking-test files.

4.4 A six-node braking test

The end-to-end example uses six complete nodes (firmware V1.0.5.6kp, 500 sps, 0 dB) that recorded for about 30 minutes on 31 March 2023 (01:28–02:00 UTC) along a road beside the sports oval in Sandy Bay, Hobart, Tasmania. The source was a Toyota Land Cruiser HJ60 braking repeatedly, about ten times; the exact times were not noted. A laughing baby rode in the back seat but was not detected. The nodes formed a group of three in the south-west (453004362, 453009194 and 453010047, 5–21 m apart), one node 41 m to the north-east (453010029) and a pair (453010077 and 453010167, 12 m apart) about 110 m from the first group (Fig. 7). The raw DLD files of all nodes are included in the repository (about 45 MB, Git LFS).

Positions from the DLD tags place the nodes and show that five of them were carried 70–230 m while still recording after 01:51. A 30-m radius around the north-east pair and a polygon around the south-west group select the expected nodes (Fig. 7; Listing 2).

Figure 8 shows results from the raw files.

- **Envelopes** show regular vehicle passes between 01:34 and 01:50 and the much stronger signals of handling afterwards (Fig. 8a).
- **Exported record section.** The Z channels exported as MiniSEED (codes ?? .DPZ, polarity inverted) show the passing vehicle node by node (Fig. 8b). The spectrogram has a broadband burst at closest approach and gliding tonal lines from the engine and drivetrain (Fig. 8c).
- **Events and speeds.** Peaks of the envelope at least eight times the background, seen by both nodes of a pair within 5 s, give 9 and 24 events at the two pairs. Matching events between the pairs gives 14 passes with a median apparent speed of 41 km/h (Fig. 8d). This is consistent with a car slowing, stopping and pulling away.
- **Spectra.** The events raise the spectrum by two to three orders of magnitude between 5 and 100 Hz (Fig. 8e).

Break test, Sandy Bay, Hobart, 31 Mar 2023



Figure 7 Braking-test nodes on 31 March 2023 positioned from the DLD tags, with tag tracks showing the nodes being carried away at the end. Dashed: a 30-m radius selection (yellow) and a polygon selection (cyan); selected nodes have white outlines. Imagery: Esri World Imagery via contextily ([Arribas-Bel et al., n.d.](#)).

The exported MiniSEED and SEG-Y samples equal -1 times the raw DLD counts exactly, and their start times equal the TOW-derived DLD times.

5 Performance

Table 2 lists execution times on a two-core 2.9-GHz Intel Xeon virtual machine (Python 3.14, ObsPy 1.5.1, pandas 3.0). Reading a 31-minute, 500-sps DLD component (2.8 MB) takes 77 ms, about 24 000 times faster than real time. A 60-s window takes about the same, because the time tags of the whole file are decoded first. One day of continuous 1000-sps data is about 265 MB per component in DLD and is read in about 7 s at the same rate. Log parsing is the slowest step, at about 1.7 s per two-week log with 9900 records. It runs once per node, and the deployment and waveform index tables can be cached as CSV.

Operation	Time
DLD header	0.06 ms
DLD time tags (924 blocks)	36 ms
DLD full component (924 000 samples)	77 ms
DLD 60-s window	66 ms
Log, 2.1 MB, 9920 records	1.7 s
Deployments from two logs	3.5 s
Deployments from 18 DLD files	0.58 s
Waveform index, 18 DLD files	1.1 s

Table 2 Median execution times (5 repetitions; 3 for the last four rows).

6 Limitations

The DLD description is derived from files of four firmware versions. Other versions may differ; `read_dld` checks the magic string and the presence and spacing of the time tags, and raises an error if they are not as expected. Three conventions remain to be confirmed:

- the block convention of the time tags;

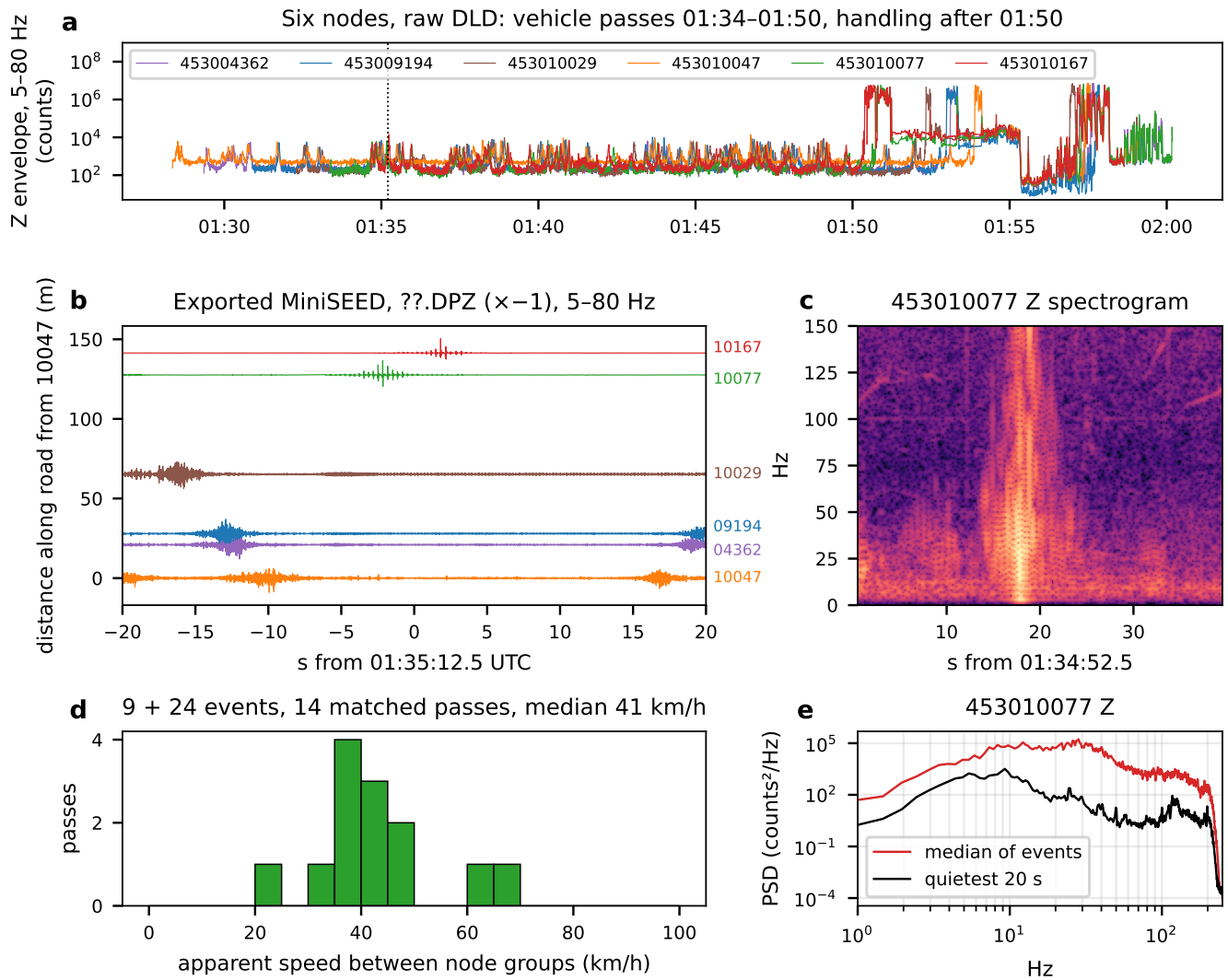


Figure 8 Braking test. (a) Vertical-component envelopes (5–80 Hz) of all six nodes read from the raw DLD files; dotted line: the event in (b, c). (b) Record section of the exported MiniSEED (polarity corrected), ordered by distance along the road. (c) Spectrogram of 453010077. (d) Apparent speeds from the travel time between the two node groups. (e) Median spectrum of the events and the quietest 20 s at 453010077.

• the mapping of the logged channels 1–3 to the X, Y and Z axes;

• the meaning of eCompass North.

The first needs one comparison with a SoloLite export, which the package supports, and the others need controlled orientation tests. We have not handled the BD3C-5 broadband variant, for which AusPass reports no polarity inversion (AusPass, 2025a); it is excluded from the automatic polarity flip by its device type. The pulse-test model (Eq. 4) is fitted to the switch-off after a negative current. After a positive current it gives about 5.35 Hz, which suggests a non-linear suspension at larger drive.

7 Conclusions

node_toolbox reads all files written by SmartSolo IGU-16HR nodes with open-source Python and connects them to standard seismological formats and tools. It makes three practical points explicit that are easy to get wrong:

• The logged sample rate is a sample interval.

- The time text in raw data tags is 2 s late until the receiver knows the leap seconds, whereas the GPS time of week is reliable.

- IGU-16HR data need a polarity inversion, and raw counts include the preamplifier gain.

The deployment logic keeps each power-up separate and places it at the first stable GPS position, which protects against cold-start errors and moved nodes. The state-of-health and quality-control tools turn the logs, which are rarely used, into information for field crews and station metadata. The code, tests, example data and tutorials are openly available, and we welcome contributions of files from other firmware versions.

Acknowledgements

We thank AusPass for publishing their SmartSolo polarity and response tests. Part of the code and text was drafted with the assistance of an AI tool (Perplexity Computer) and checked by the author.

Data and code availability

The code, unit tests, tutorial notebooks, figure scripts (`paper/make_figures.py`) and example data are available at https://github.com/TobbeTripitaka/node_toolbox under the MIT licence (Stål, 2026). A versioned snapshot will be archived on Zenodo upon acceptance (DOI to be added). The repository contains the state-of-health logs and pulse-test files of the two Antarctic nodes and the complete files, including raw DLD data, of the six braking-test nodes (Git LFS). The user manual consists of the repository README, the module docstrings, `docs/DLD_FORMAT.md`, `docs/FINDINGS.md` and the notebooks. Basemap imagery in Fig. 7 is Esri World Imagery (sources: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP and the GIS User Community).

Competing interests

The authors have no competing interests. `node_toolbox` is independent of, and not endorsed by, DTCC or SmartSolo.

References

- Alken, P., Thébault, E., Beggan, C. D., et al. International Geomagnetic Reference Field: the thirteenth generation. *Earth, Planets and Space*, 73:49, 2021. doi: 10.1186/s40623-020-01288-x.
- Arribas-Bel, D. et al. contextily: context geo tiles in Python. Software, n.d. <https://github.com/geopandas/contextily>. Accessed 2026-10-05.
- AusPass. AusPass Data: SmartSolo node polarity issues. AusPass: the Australian Passive Seismic Server, Research School of Earth Sciences, Australian National University, 2025a. <https://auspass.edu.au/xwiki/bin/view/Data/AusPass%20Data/>. Accessed 2026-10-05.
- AusPass. SmartSolo Nodes. AusPass: the Australian Passive Seismic Server, Research School of Earth Sciences, Australian National University, 2025b. <https://auspass.edu.au/xwiki/bin/view/Instrumentation/SmartSolo%20Nodes/>. Accessed 2026-10-05.
- Beyreuther, M., Barsch, R., Krischer, L., Megies, T., Behr, Y., and Wassermann, J. ObsPy: A Python Toolbox for Seismology. *Seismological Research Letters*, 81(3):530–533, 2010. doi: 10.1785/gssrl.81.3.530.
- DTCC. *SmartSolo Hardware User Manual, version 2.0*. DTCC (SmartSolo), n.d. https://nappe.wustl.edu/smartsolo/files/SmartSolo_Hardware_User_Manual_EN_V2.0.pdf. Accessed 2026-10-05.

- 319 Harris, C. R., Millman, K. J., van der Walt, S. J., et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-
320 2649-2.
- 321 Hunter, J. D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.
- 322 International Federation of Digital Seismograph Networks. *SEED Reference Manual, Standard for the Exchange of Earthquake Data, SEED*
323 *Format Version 2.4*. FDSN, IRIS and USGS, 2012. https://www.fdsn.org/pdf/SEEDManual_V2.4.pdf.
- 324 International Federation of Digital Seismograph Networks. *FDSN StationXML schema documentation*. FDSN, n.d. [https://docs.fdsn.org/](https://docs.fdsn.org/projects/stationxml/en/latest/)
325 [projects/stationxml/en/latest/](https://docs.fdsn.org/projects/stationxml/en/latest/). Accessed 2026-10-05.
- 326 Jordahl, K., Van den Bossche, J., Fleischmann, M., et al. geopandas/geopandas: v0.8.1. Zenodo, 2020. doi: 10.5281/zenodo.3946761.
- 327 Karplus, M. and Schmandt, B. Preface to the Focus Section on Geophone Array Seismology. *Seismological Research Letters*, 89(5):1597–
328 1600, 2018. doi: 10.1785/0220180212.
- 329 Krischer, L., Megies, T., Barsch, R., Beyreuther, M., Lecocq, T., Caudron, C., and Wassermann, J. ObsPy: a bridge for seismology into the
330 scientific Python ecosystem. *Computational Science & Discovery*, 8(1):014003, 2015. doi: 10.1088/1749-4699/8/1/014003.
- 331 McKinney, W. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61,
332 2010. doi: 10.25080/Majora-92bf1922-00a.
- 333 Ringler, A. T., Anthony, R. E., Karplus, M. S., Holland, A. A., and Wilson, D. C. Laboratory Tests of Three Z-Land Fairfield Nodal 5-Hz, Three-
334 Component Sensors. *Seismological Research Letters*, 89(5):1601–1608, 2018. doi: 10.1785/0220170236.
- 335 SEG Technical Standards Committee. SEG-Y_r2.0: SEG-Y revision 2.0 Data Exchange format. Technical report, Society of Exploration Geo-
336 physicists, 2017. https://library.seg.org/pb-assets/technical-standards/seg_y_rev2_0-mar2017-1686080998003.pdf.
- 337 Stål, T. node_toolbox: open-source Python tools for SmartSolo node data. Software, GitHub repository, 2026. [https://github.com/](https://github.com/TobbeTripitaka/node_toolbox)
338 [TobbeTripitaka/node_toolbox](https://github.com/TobbeTripitaka/node_toolbox). Zenodo DOI to be minted on acceptance.
- 339 Virtanen, P., Gommers, R., Oliphant, T. E., et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:
340 261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- 341 Zeckra, M., Van Noten, K., and Lecocq, T. Sensitivity, Accuracy and Limits of the Lightweight Three-Component SmartSolo Geophone Sensor
342 (5 Hz) for Seismological Applications. EarthArXiv preprint, 2022. doi: 10.31223/X5F073.